

Competitive Programming

Section 52 Editorial: Game Theory

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Nim Winner	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Subtraction Game	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. DAG Game	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Nim XOR. Let x be the xor of all pile sizes. If $x = 0$, every move changes one pile and makes the xor nonzero, because changing one operand of an xor-zero collection cannot keep the xor zero. If $x \neq 0$, choose a pile whose highest set bit matches the highest set bit of x , and reduce it to $\text{pile} \oplus x$, which is smaller. The new global xor is zero. Thus xor-zero positions are losing and nonzero positions are winning.

B. Subtraction Game DP. Let $\text{win}[i]$ mean the current player wins with i stones. The base $\text{win}[0] = \text{false}$ because no move is possible. For $i > 0$, $\text{win}[i]$ is true if there exists an allowed move $d \leq i$ with $\text{win}[i-d] == \text{false}$; otherwise every move gives the opponent a winning state, so $\text{win}[i]$ is false. This is induction on the number of stones.

C. Grundy on a DAG. A terminal node has Grundy value 0. For every other node, compute the mex of the Grundy values of its outgoing neighbors. In a DAG, reverse topological order guarantees children are known before parents. A start node is winning iff its Grundy value is nonzero, by the same mex argument used for impartial games in the lesson.

Common pitfalls. Do not confuse “has a move” with “is winning”; a move must go to a losing state. For mex, only reachable child values matter. For Nim, the winning move reduces one pile, never increases it.

Every problem is a normal-play impartial game.

- A uses the Nim theorem: xor zero is losing, xor nonzero is winning.
- B computes $\text{win}[i]$ for a subtraction game from smaller piles to larger piles.
- C computes Grundy numbers on a DAG by taking the mex of successor Grundy numbers.

The shared proof is recursive: a state is winning if it has at least one move to a losing state; it is losing if every move goes to a winning state.

2 A. Nim Winner

2.1 Restatement

For each Nim position, print WIN or LOSE.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
```

```
n=data[idx]; idx+=1; x=0
for v in data[idx:idx+n]: x^=v
idx+=n; out.append("WIN" if x else "LOSE")
print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Subtraction Game

3.1 Restatement

Given total stones and allowed moves, print whether the first player wins.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; moves=data[2:]; win=[False]*(n+1)
    for i in range(1,n+1): win[i]=any(i>=x and not win[i-x] for x in moves)
```

```
    print("WIN" if win[n] else "LOSE")  
if __name__=="__main__": main()
```

4 C. DAG Game

4.1 Restatement

For each start vertex in a DAG, print whether it is winning.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,q;if(!
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,q=data[0],data[1],data[2]; idx=3; g=[] for _ in range(n)
    for _ in range(m): g[data[idx]-1].append(data[idx+1]-1); idx+=2
```

```

gr=[0]*n
for u in range(n-1,-1,-1):
    seen={gr[v] for v in g[u]}; x=0
    while x in seen: x+=1
    gr[u]=x
out=["WIN" if gr[data[idx+i]-1] else "LOSE" for i in range(q)]
print("\n".join(out))
if __name__=="__main__": main()

```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.